Agenda, day 2: Loops, lists, and tuples

- Q&A
- Loops
  - What are they?
  - `for`
  - Where's the index?
  - Repeating a number of times
  - `while`
  - Controlling our loops with `break` and `continue`
- Lists
  - What is a list?
  - How is it similar to a string? How is it different?
  - Lists are mutable
  - Methods to modify our lists -- `list.append`, `list.extend`, `list.pop`
  - Looping over lists
- Strings into lists, and back
  - Turning a string into a list of strings with `str.split`
  - Turning a list of strings into a single string with `str.join`
- Tuples
  - Tuple basics -- creating, retrieving, what to do with them, why they exist
  - Tuple unpacking -- what it is, and how to use it

Loops

Let's say that I have a string, `'abcd'`, and I want to print each of the characters in the string. I can do that:

In [1]:
```
s = 'abcd'  # assign the string to a variable

print(s[0])
print(s[1])
print(s[2])
print(s[3])
```

```
a
b
c
d
```

What's the problem?

We're repeating ourselves. The same code (more or less) is taking place on lines 3, 4, 5, and 6.

We generally want to avoid such repetition. In fact, there's a famous rule, DRY -- don't repeat yourself!

What's wrong with repeating yourself?

- More work writing it
- More work reading it -- just reading code is hard, and reading repeated code is extra-hard
- More work maintaining it -- if we find a bug in the code, then we have to fix it wherever that code appears. That can be a one-time thing if the code is centralized/not repeated, or it can be a day-long event, tracking down where the code exists.

One of the best ways to "DRY up" your code is to use a *loop*, where code repeats a number of times. Each repetition, known as an *iteration*, can be slightly differnt from the others.

In Python, we have exactly two types of loops:

- `for` loops, more common and go through every element of a string or similar sequence
- `while` loops, which are less common, and are very similar to `if` statements that repeat.

In [2]:
```
# simple loop to do what we did above

s = 'abcd'

for one_character in s:
    print(one_character)
```

```
a
b
c
d
```

Syntax of a `for` loop

1. We start with the keyword `for`
2. Then we have the *loop variable*, in this case, `one_character`. This is where the loop assigns the new value with each iteration.
3. Then we have the keyword `in`
4. Then we have the value over which we want to iterate. Here, it'll be `s`, the variable, which refers to a string.

5. At the end of the line, we have a `:`, indicating that the next line needs to be indented.
6. The next line starts an indented block. Its execution will continue until the indentation ends (or until the program ends)
7. The loop will execute once for each character in `s`, until we get to the end. At that point, the loop exits silently and continues with the rest of the program.

Conversation of a `for` loop

- `for` turns to the value at the end of the line (`s`, in this case) and asks: Are you iterable? Meaning, do you know how to behave in a `for` loop?
  - If not, then the loop exits with an error
- If the value is iterable, then `for` asks for its next value
  - If we've reached the end of the values, then the loop exits and we continue with the code after the loop.
- The "next value" is assigned to the loop variable (in this case, `one_character`.
- Then the body of the loop executes with the defined loop variable.
- When we get to the end of the loop body, we go back and ask (again) for the next value. It'll either be assigned to the loop variable, or the loop will be told that there are no more values, and we should exit

Notice a few things here

- The name of the loop variable has *NOTHING* to do with what values we get.
- We don't know in advance how many iterations there will be, or what values we'll get

Exercise: Vowels, digits, and others

We're going to ask the user to enter a text string. We'll then iterate over that string, one character at a time, and check:

- Is it a vowel? (a, e, i, o. u)
- Is it a digit? (0-9)
- Is it something else?

To do:

1. Define three variables, `vowels`, `digits`, and `others`, and set them to all be 0.
2. Ask the user, with `input`, to enter some text.
3. Use a `for` loop to go through the text, one character at a time.

4. Print each of the counters.

If I enter:

```
Text: hello!! 123
vowels: 2
digits: 3
others: 6
```

In [3]:
```
vowels = 0
digits = 0
others = 0

text = input('Enter text: ').strip()   # assign the user's input to "text",

for one_character in text:
    if one_character in 'aeiou':    # is one_character a vowel?
        vowels += 1                 # ... if so, then add 1 to vowels
    elif one_character.isdigit():   # is one_character a digit, 0-9?
        digits += 1                 # ... if so, then add 1 to digits
    else:
        others += 1                 # neither a vowel nor a digit? Add 1 to

print(f'vowels: {vowels}')
print(f'digits: {digits}')
print(f'others: {others}')
```

```
vowels: 2
digits: 3
others: 6
```

What if I just want to repeat something?

Let's say that I'm in a great mood, and want to celebrate via code:

In [4]:
```
print('Hooray!')
print('Hooray!')
print('Hooray!')
```

```
Hooray!
Hooray!
Hooray!
```

In [5]:
```
# the above code works... but it's not very DRY. how can we get this to not

# what if I iterate over the number 3?

for one_item in 3:
    print('Hooray!')
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[5], line 5
      1 # the above code works... but it's not very DRY. how can we get this
to not repeat?
      2
      3 # what if I iterate over the number 3?
      4
----> 5 for one_item in 3:
      6     print('Hooray!')

TypeError: 'int' object is not iterable
```

The solution: `range`

We can invoke `range(n)`, which is designed to be used in a `for` loop:

In [6]:
```
for one_item in range(5):   # range(5) iterates 5 times
    print('Hooray!')
```

```
Hooray!
Hooray!
Hooray!
Hooray!
Hooray!
```

What is `range`? What are we getting with each iteration?

In [7]:
```
for one_item in range(5):   # range(5) iterates 5 times
    print(f'[{one_item}] Hooray!')
```

```
[0] Hooray!
[1] Hooray!
[2] Hooray!
[3] Hooray!
[4] Hooray!
```

Iterating over `range`

When we iterate over `range`, we get integers:

- Starting with the initial number (default is 0)
- Up to and *not* including the final number (which is the only mandatory argument for range)

`range(n)` will give you, on each iteration, the next number, until it reaches `n-1`, which is the final value it returns.

In [8]:
```
# Back to vowels, digits, others briefly:

vowels = 0
digits = 0
others = 0

text = input('Enter text: ').strip()   # assign the user's input to "text",

for one_character in text:
    if one_character in ['a', 'e', 'i', 'o', 'u']:    # this still works! Bu
        vowels += 1                 # ... if so, then add 1 to vowels
    elif one_character.isdigit():   # is one_character a digit, 0-9?
        digits += 1                 # ... if so, then add 1 to digits
    else:
        others += 1                 # neither a vowel nor a digit? Add 1 to

print(f'vowels: {vowels}')
print(f'digits: {digits}')
print(f'others: {others}')
```

```
vowels: 2
digits: 3
others: 6
```

What's the difference?

If you define `'aeiou'`, then you have defined a string.

If you define `['a', 'e', 'i', 'o', 'u']`, then you have defined a *list*, a completely different data type, but one that has a lot of ties to strings and a lot of overlapping functionality.

It's a bit faster and smaller to use a string in this case. But we'll talk more about lists soon.

In [9]:
```
for counter in range(5):
    print(counter)
```

```
0
1
2
3
4
```

In [10]:
```
for counter in range(100):
    print(counter, end=' ')
```

```
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 5
4 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79
80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99
```

Exercise: Triangle

1. Ask the user to enter a 1-character string. Assign to `char`.
2. Print a triangle with this character, string with 0 on the first line and 9 on the final line.

Note: If you try to add an integer and string, you'll get an error. But multiply them, and it works just fine.

In [11]:
```
'a' + 5
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[11], line 1
----> 1 'a' + 5

TypeError: can only concatenate str (not "int") to str
```

In [12]:
```
'a' * 5
```

Out[12]:
```
'aaaaa'
```

In [15]:
```
char = input('Enter a character: ').strip()

for index in range(10):   # why 10? because range always goes up to n-1, whe
    print(index * char)   # create a string, index*char, and print it
```

```
*
**
***
****
*****
******
*******
********
*********
```

In [16]:
```
print('-' * 60)
print('Very important comment here')
print('-' * 60)
```

```
------------------------------------------------------------
Very important comment here
------------------------------------------------------------
```

Next up

1. Indexes (or the fact that we don't have them)
2. Controlling our loop
3. `while` loops, and how they're similar/different

Indexes

In C (and similar languages), a `for` loop has three parts:

- Starting integer (usually 0)
- An action to do with each iteration (e.g., `i = i + 1`)
- Ending condition (e.g., `i < 5`)

As you can see, a `for` loop in such a language is all about the numbers. You can go down, you can go up, you can go up by large increments, but it's all about the numbers.

You then use the index to retrieve whatever character you want from the string. There's no such thing as iterating over the characters in a string.

In Python, we say: Why, if we want to iterate over the characters of a string, do we need a separate data structure (i.e., an integer)? Just give me the characters, one at a time!

That's why our `for` loop looks and works the way it does.

However... there are times when you'll want to use the index, either for calculations or for displaying the index next to each character. How can we do that?

In [18]:
```
# Option 1: Pretend we're in C
# warning: very ugly code ahead

s = 'abcd'

for i in range(len(s)):    # get the length of s, then run range on that
    print(f'[{i}] {s[i]}')
```

```
[0] a
[1] b
[2] c
[3] d
```

In [20]:
```
# Option 2: Set the index manually
# here, I'm getting each character -- and as each character comes to us from

i = 0

for one_character in s:
    print(f'[{i}] {one_character}')
    i += 1    # this adds 1 to i
```

```
[0] a
[1] b
[2] c
[3] d
```

In [21]:
```python
# Option 3: Use the "enumerate" function that comes with Python
# enumerate gets, as input, the string (or other iterable you want to get th

for i, one_character in enumerate(s):     # yes, we're iterating over 2 valu
    print(f'[{i}] {one_character}')
```

```
[0] a
[1] b
[2] c
[3] d
```

Exercise: Powers of 10

If I have a decimal number, such as 3579, then I can rewrite it as the sum of powers of 10:

- 3 * 10**3
- 5 * 10**2
- 7 * 10**1
- 9 * 10**0 # anything to the 0 power is 1

I want you to ask the user to enter a number. (Assume that they do.) Print each of the digits in the same format as I just did in my example.

In [31]:
```python
number = input('Enter a number: ').strip()

if number.isdigit():
    for index, one_digit in enumerate(number):   # iterate over the number (
        exponent = len(number) - index - 1       # calculate the exponent --
        print(f'{one_digit} * 10**{exponent}')   # print things using an f-s

else:
    print(f'{number} is not numeric; try again.')
```

```
4 * 10**1
6 * 10**0
```

Controlling our loops

So far, we've seen that we can iterate over all of the characters in a string with a `for` loop. What if we want to iterate over only the first half, exiting when the index is before the end?

Right now, we have two ways to do that:

1. We can iterate over a slice (i.e., a partial string)
2. We can get the index, and only print if the index is low

Neither of these really gets to the heart of the problem: Sometimes, when we're iterating in a `for` loop, we discover that we can/should go onto the nextr iteration, or we should exit the loop entirely.

When would these be appropriate? Often:

- We need to exit the loop immediately, if we have accomplished our goal -- this is done with `break`
- We might not want to exit the loop, but we might want to exit the current iteration, if it's clearly unnecessary. If the current input from the user is completely invalid, then we should move onto the next one, not go through the entire loop body. This is done with `continue`

In [32]:
```
# end the loop with break

s = 'abcde'
look_for = 'c'

print('Before')
for one_character in s:
    if one_character == look_for:
        break   # end the loop right here!

    print(one_character)
print('After')
```

```
Before
a
b
After
```

In [33]:
```
# end the iteration (but not the loop) with continue

s = 'abcde'
look_for = 'c'

print('Before')
for one_character in s:
    if one_character == look_for:
        continue   # end this iteration, but keep going through the rest of

    print(one_character)
print('After')
```

```
Before
a
b
d
e
After
```

Exercise: Sum digits

1. Ask the user to enter a string with digits.
2. Set `total` to 0.
3. Go through each character in the user's input string.
   - If it's a `.`, then stop the loop entirely, and print `total`
   - If it's a non-digit (other than `.`), then scold the user and go on
   - If it's a digit, then add to `total`.
4. Print `total`.

In [34]:
```
total = 0
digits = input('Enter some digits: ').strip()

for one_character in digits:
    if one_character == '.':
        print(f'Ending; total is {total}')
        break    # end the loop!

    if one_character.isdigit():
        total += int(one_character)
        print(f'Added {one_character}; total is now {total}')

    else:
        print(f'{one_character} is not a digit; ignoring.')
        continue
```

```
Added 2; total is now 2
Added 4; total is now 6
Added 6; total is now 12
a is not a digit; ignoring.
b is not a digit; ignoring.
Ending; total is 12
```

In [35]:
```
total = 0
digits = input('Enter some digits: ').strip()

for one_character in digits:
    if one_character == '.':
        print(f'Ending; total is {total}')
        break    # end the loop!

    if not one_character.isdigit():
        print(f'{one_character} is not a digit; ignoring.')
        continue

    total += int(one_character)
    print(f'Added {one_character}; total is now {total}')
```

```
Added 2; total is now 2
Added 4; total is now 6
Added 6; total is now 12
a is not a digit; ignoring.
b is not a digit; ignoring.
Ending; total is 12
```

`while` loops

In a `for` loop, we start at the beginning, and go through each element of the sequence. We either get to the end or invoke `break` to stop. But the idea in a `for` loop is that we're iterating over each element, one at a time. We stop at the end.

By contrast. `while` loops are meant for situations when we don't know how many iterations we'll need. Maybe we'll need many, and maybe not that many. Maybe none at all.

A `while` loop is basically an `if` statement whose body repeats so long as the condition is `True`.

- An `if` looks to its right, and evaluates the value there. If the value is `True`, then the block runs.
- In the case of `while`, the same thing happens. The only difference is that after the block runs, we go *back* to the `while`, and it runs again.

In [36]:
```
# example while loop

x = 5

while x > 0:
    print(x)
    x -= 1   # x = x - 1
```

```
5
4
3
2
1
```

Note that both `break` and `continue` work just fine in both `while` and `for` loops.

Exercise: Sum to 100

1. Set `total` to 0
2. Ask the user to enter a number.
   - If it's not a valid number, scold the user.
   - If it's the empty string (''), then stop the program

- Convert the number to an `int`, and add to `total`.

3. When `total` is >= 100, stop and print `total`.

In [38]:
```
total = 0

while total < 100:
    s = input('Enter a number: ').strip()

    if not s.isdigit():
        print(f'Not a number!')
        continue

    if s == '':
        break

    total += int(s)
    print(f'{total=}')
```

```
total=30
total=50
Not a number!
total=51
total=52
total=53
total=54
total=55
total=135
```

Next up

1. Lists!
2. Converting strings to lists (and back)
3. Tuples + unpacking

Lists

A list is another data type (along with integers, floats, and strings). It, like `str`, is a member of the "sequence" family, which means that there are many similarities between strings and lists.

A list is a *container*. It can contain any number of values, and those other values can be of any type at all. It is, however, traditional to have all of the elements of a list be of the same type.

Where are lists used?

- A list of employees
- A list of sales records

- A list of vehicles in the parking lot
- A to-do list
- A list if IP addresses we want to block from our server

List syntax

To create a list, we use `[]`:

- Put each element inside of the `[]`, separated with `,` from other elements
- If you have only one element, then no `,` is needed
- If you have the "empty list," then still no `,` is needed

In [39]:
```
mylist = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]

type(mylist)
```

Out[39]:
```
list
```

Don't call your list variable `list`. It will work, but will also gum some things up.

In [40]:
```
len(mylist)  # just like a string
```

Out[40]:
```
10
```

In [41]:
```
mylist[0]   # retrieve the first element, like a string with [0]
```

Out[41]:
```
10
```

In [42]:
```
mylist[1]
```

Out[42]:
```
20
```

In [43]:
```
# final element -- use [-1]

mylist[-1]
```

Out[43]:
```
100
```

In [48]:
```
# slicing of lists!

mylist[3:7]   # from index 3 up to (and not including) index 7
```

Out[48]:
```
[40, 50, 60, 70]
```

In [45]:
```
# search in a list with "in"

50 in mylist
```

Out[45]:
```
True
```

In [46]:
```
1234 in mylist
```

Out[46]:
```
False
```

In [47]:
```
# loop on a list

for one_item in mylist:
    print(one_item)
```

```
10
20
30
40
50
60
70
80
90
100
```

We can change a list!

If a data structure is *mutable* in Python, we can change its value. We cannot change the value of an integer, but we can get a new number back. The same is (strangely) true for strings in Python -- we cannot change its value, but we can get a new one back.

But in the case of a list, we can indeed change it!

- Replace an existing value with a new one
- Add new elements to the list (typically at the end)
- Remove existing elements from the list

In [49]:
```
# to replace an existing value, just assign to the index

mylist
```

Out[49]:
```
[10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
```

In [50]:
```
mylist[3] = '!!!'
mylist
```

Out[50]:
```
[10, 20, 30, '!!!', 50, 60, 70, 80, 90, 100]
```

In [51]:
```
# if you try to assign outside of the index boundaries, you'll get an error
mylist[1000] = '????'
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[51], line 2
      1 # if you try to assign outside of the index boundaries, you'll get a
n error
----> 2 mylist[1000] = '????'

IndexError: list assignment index out of range
```

In [52]:
```
# add new elements, then I can use list.append or list.extend

mylist.append(110)  # we added 110
```

In [53]:
```
mylist.extend([9,8,7])  # extend runs a for loop on the argument, and append
```

In [54]:
```
mylist
```

Out[54]:
```
[10, 20, 30, '!!!', 50, 60, 70, 80, 90, 100, 110, 9, 8, 7]
```

In [55]:
```
mylist.append([6,5,4])
mylist
```

Out[55]:
```
[10, 20, 30, '!!!', 50, 60, 70, 80, 90, 100, 110, 9, 8, 7, [6, 5, 4]]
```

In [56]:
```
# remove an element with list.pop - this removes the final element and retur
# or you can invoke list.pop with an integer index, and it'll remove+return
```

In [57]:
```
mylist.pop()
```

Out[57]:
```
[6, 5, 4]
```

In [58]:
```
mylist.pop()
```

Out[58]:
```
7
```

In [59]:
```
# you can provide the index you want

mylist.pop(2)
```

Out[59]:
```
30
```

In [60]:
```
mylist
```

Out[60]:
```
[10, 20, '!!!', 50, 60, 70, 80, 90, 100, 110, 9, 8]
```

Exercise: Vowels, digits, and others (list edition)

1. Assign `[]` (the empty list) to `vowels`, `digits`, and `others`.
2. Ask the user to enter some text.

3. Go through that text, one character at a time, and classify it as vowel/digits/other.
4. Append that character to the appropriate list.
5. Print the three variables.

In [61]:
```
vowels = []
digits = []
others = []

text = input('Enter text: ').strip()

for one_character in text:
    if one_character in 'aeiou':          # if one_character is a vowel..
        vowels.append(one_character)      # ... add it to the end of the "vow
    elif one_character.isdigit():         # if one_character is a digit...
        digits.append(one_character)
    else:
        others.append(one_character)

print(f'vowels = {vowels}')
print(f'digits = {digits}')
print(f'others = {others}')
```

```
vowels = ['e', 'o']
digits = ['1', '2', '3']
others = ['h', 'l', 'l', '!', '!', ' ']
```

Exercise: Multiples of

1. Define three empty lists, assigned to `mult2`, `mult3`, and `mult5`.
2. Ask the user to enter a number.
3. Turn that number into an integer with `int`. Use `range` to iterate from 0 through that number.
4. Check if each iteration's number is divisible by 2, 3, and 5:
   - If it's divisible by 2, append to `mult2`
   - If it's divisible by 3, append to `mult3`
   - If it's divisible by 5, append to `mult5`
   - It's OK for a number to be on more than one list. (10 will be on `mult2` and also `mult5`.

Remember that if you say `n % 2` for any integer `n`, and you get 0, that means there is no remainder when you divide n by 2.

In [63]:
```
mult2 = []
mult3 = []
mult5 = []

number = input('Enter a number: ').strip()

for n in range(int(number)):
```

```
    if n % 2 == 0:  # evenly divisible by 2,4,6,8?
        mult2.append(n)
    if n % 3 == 0:  # evenly divisible by 3,6,9,12,etc.
        mult3.append(n)
    if n % 5 == 0:   #evenly divisible by 5
        mult5.append(n)

print(f'mult2 = {mult2}')
print(f'mult3 = {mult3}')
print(f'mult5 = {mult5}')
```

```
mult2 = [0, 2, 4, 6]
mult3 = [0, 3, 6]
mult5 = [0, 5]
```

Next up

1. `str.split` and `str.join`
2. Tuples, unpacking, etc.

Off by one errors

One of the most common mistakes in programming is an "off by one error." This often crops up in languages where we start to number the index at 0.

It then becomes an issue of "up to and including" or "up to and not including," and you have to specify it everywhere.

And you'll still get it wrong sometimes.

We've seen that strings and lists have a lot in common, thanks to them both being "sequences." Python was designed to make it easy to learn to program.

However, strings and lists aren't the same. Just as we sometimes (often) need to convert strings to ints, and also ints to strings, how can we convert our string to a list?

In [64]:
```
s = '1234'

int(s)   # we invoke int on s, and get back a new integer
```

Out[64]:
```
1234
```

In [65]:
```
i = 1234

str(i)   # we invoke str on i, and get back a new string
```

Out[65]: '1234'

In [66]:
```
# maybe we can invoke list on a string, and get back a list?

s = 'this is important'
list(s)  # I do get a list...
```

Out[66]:
```
['t',
 'h',
 'i',
 's',
 ' ',
 'i',
 's',
 ' ',
 'i',
 'm',
 'p',
 'o',
 'r',
 't',
 'a',
 'n',
 't']
```

In [67]:
```
# what would we want to do?
# assume this:

info = 'Reuven,Lerner,reuven@lernerpython.com'

# our normal, natural, human way of interpeting this is that there are 3 fie

info.split(',')  # split is a string method that returns a list of strings
```

Out[67]: ['Reuven', 'Lerner', 'reuven@lernerpython.com']

In [68]:
```
s = 'abcd::ef::ghij'

s.split('::')  # a 2-character delimiter works, too
```

Out[68]: ['abcd', 'ef', 'ghij']

In [69]:
```
s.split('d')  # weird, but Python will do it
```

Out[69]: ['abc', '::ef::ghij']

In [70]:
```
# one case is particularly common -- splitting a sentence into words

s = 'this is an example sentence'

s.split(' ')
```

Out[70]: ['this', 'is', 'an', 'example', 'sentence']

In [71]:
```
s = 'this     is    an    example       sentence'

s.split(' ')
```

Out[71]:
```
['this',
 '',
 '',
 '',
 '',
 'is',
 '',
 '',
 'an',
 '',
 '',
 '',
 'example',
 '',
 '',
 '',
 '',
 '',
 '',
 '',
 'sentence']
```

If there are two or more space characters in a row, str.split(' ') will produce an empty string as an element.

The solution? Don't pass an argument to str.split. It will then treat all whitespace, of any length, as a single separator.

In [73]:
```
s.split()  # no argument!
```

Out[73]:
```
['this', 'is', 'an', 'example', 'sentence']
```

Exercise: Sum numbers (not the same)

1. Set `total` to 0.
2. Ask the user to enter a bunch of numbers, separated by spaces.
3. Go through each number, and add to `total`
   - If you cannot turn it into a number, let the user know.
4. Print `total`

In [77]:
```
total = 0

text = input('Enter numbers, separated by spaces: ').strip()

for one_number in text.split():   # split returns a list of strings
    if not one_number.isdigit():  # if we cannot turn the current "number" i
        continue
```

```
    n = int(one_number)
    total += n
    print(f'Added {one_number}; total is now {total}')
```

```
Added 10; total is now 10
Added 20; total is now 30
Added 30; total is now 60
Added 40; total is now 100
```

The opposite of `str.split`

We've now seen:

- `str.split` is a string method
- It always returns a list of strings, even if that's an empty list
- If you invoke it with an argument, that is used as the delimiter
- If you invoke it without an argument, then all whitespace, of any type and any length and any combination, is considered a delimiter.

What about the opposite?

- I have a list of strings, and want to combine them

I can do this with `str.join`. Notice that it's a string method, *not* a list method!

In [78]:
```
mylist = ['abcd', 'ef', 'ghij']

# The '*' here is the "glue," put between elements of mylist (but not on the
'*'.join(mylist)
```

Out[78]:
```
'abcd*ef*ghij'
```

In [79]:
```
'!!!'.join(mylist)
```

Out[79]:
```
'abcd!!!ef!!!ghij'
```

Exercise: Pig Latin sentence!

Last week, we created a Pig Latin translator. Now, I want you to get a sentence (all lowercase, no punctuation) from the user, and print it on the screen, in a single line, translated.

Example:

```
Enter sentence: this is the best ever
histay isway hetay estbay everway
```

In [83]:
```
sentence = input('Enter a sentence: ').strip()

for word in sentence.split():
    if word[0] in 'aeiou':
        print(word + 'way')
    else:
        print(word[1:] + word[0] + 'ay')
```

```
ellohay
outway
heretay
```

In [85]:
```
# use a list

output = []
sentence = input('Enter a sentence: ').strip()

for word in sentence.split():
    if word[0] in 'aeiou':
        output.append(word + 'way')
    else:
        output.append(word[1:] + word[0] + 'ay')

print(' '.join(output))
```

```
ellohay outway heretay
```

Tuples

Tuples are the third member of the "sequence" family, along with strings and lists.

- Strings contain characters, and are immutable
- Lists contain anything, and are mutable
- Tuples combine these -- they can contain anything, and they're immutable.

In [86]:
```
# create a tuple (normally) with ()

t = (10, 20, 30, 40, 50)

type(t)
```

Out[86]:
```
tuple
```

In [87]:
```
# tuples do all of the things lists do... with one exception -- they're immu

t[0] = '!'
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[87], line 3
      1 # tuples do all of the things lists do... with one exception -- the
y're immutable!
      2
----> 3 t[0] = '!'

TypeError: 'tuple' object does not support item assignment
```

You might remember that I earlier said that Python convention is not to have more than one value type in a single list.

Tuples are the opposite, in many ways -- if you have a tuple, then you can assume it has different types.

You can use a tuple to store things like a record/struct, or you might get them when you retrieve from a database.

Tuple unpacking

This means that on the *left* side, you have a tuple of variables. On the right side, you have a number of values, or an iterable with those values.

In [88]:
```
mylist = [10, 20, 30]

x,y,z = mylist   # the 3 elements of mylist will be assigned to x, y, and z
```

In [89]:
```
x
```

Out[89]:
```
10
```

In [90]:
```
y
```

Out[90]:
```
20
```

In [91]:
```
z
```

Out[91]:
```
30
```

In [92]:
```
for index, one_letter in enumerate('abcdef'):
    print(f'{index=}, {one_letter}')
```

```
index=0, a
index=1, b
index=2, c
index=3, d
index=4, e
index=5, f
```

In [93]:
```
for t in enumerate('abcdef'):
    print(t)
```

```
(0, 'a')
(1, 'b')
(2, 'c')
(3, 'd')
(4, 'e')
(5, 'f')
```

In [94]:
```
10,20,30
```

Out[94]: (10, 20, 30)

In [95]:
```
t = 10, 20, 30
t
```

Out[95]: (10, 20, 30)

In [96]:
```
x = 999
y = 222

x,y = y,x
```

In [97]:
```
x
```

Out[97]: 222

In [98]:
```
y
```

Out[98]: 999

Next time

1. Dictionaries
2. Files

In [ ]: